

homework5

2026-09-24

Homework 5

Question 1

a) Seed: 43251.

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = 0.594218.$$

k	Prediction
1.0000	0.8475
3.0000	0.5707
5.0000	0.6424
7.0000	0.6143
9.0000	0.6414
11.0000	0.5767
13.0000	0.5563
15.0000	0.5517
17.0000	0.5790
19.0000	0.6116
21.0000	0.5296
23.0000	0.5152
25.0000	0.5573
27.0000	0.5182
29.0000	0.5384

b) The largest numerical difference in $\text{MSE} - (\text{bias}^2 + \text{variance})$ was $1.12e - 16$.

k	Squared bias	Variance	MSE
1.0000	0.00094	0.03548	0.03643
3.0000	0.00180	0.01786	0.01967
5.0000	0.00397	0.01408	0.01805
7.0000	0.00375	0.01200	0.01575
9.0000	0.00553	0.01080	0.01633
11.0000	0.00650	0.00948	0.01599
13.0000	0.00702	0.00892	0.01593
15.0000	0.00837	0.00919	0.01756
17.0000	0.00830	0.00826	0.01656
19.0000	0.00940	0.00793	0.01733
21.0000	0.01089	0.00774	0.01863
23.0000	0.01323	0.00724	0.02047
25.0000	0.01431	0.00642	0.02073
27.0000	0.01465	0.00625	0.02090
29.0000	0.01552	0.00601	0.02152

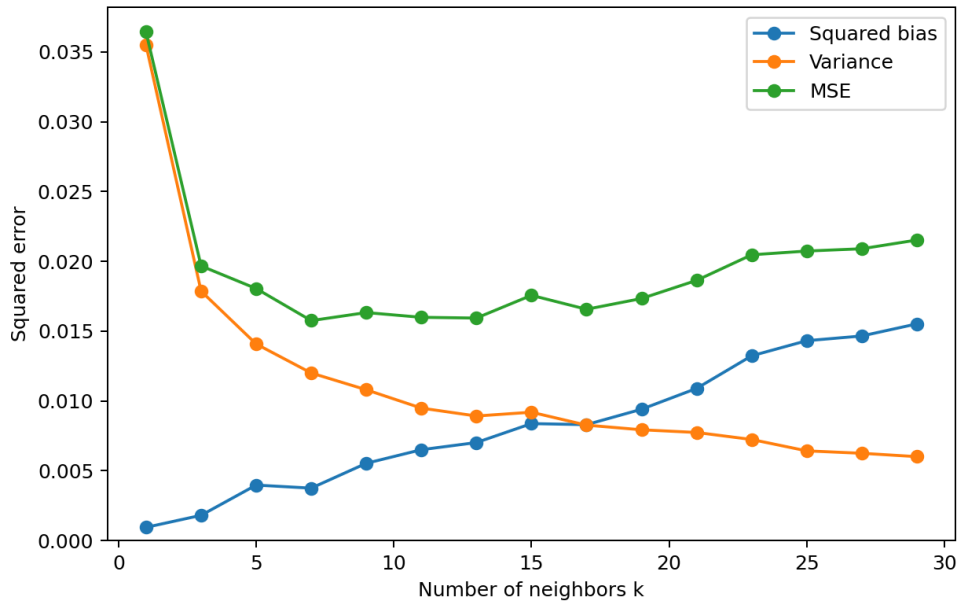


Figure 1: Squared bias, variance, and MSE across k .

As k grows, variance generally decreases while squared bias increases because more distant observations enter the neighborhood. The MSE is smallest near an intermediate k ; this is similar to ridge or lasso, where stronger regularization trades lower variance for higher bias.

```
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

np.random.seed(43251)
n, p = 400, 3
ks = np.arange(1, 30, 2)
x0 = np.array([[0.5, 0.7, 1.0]])
f0 = 0.5*x0[0,0] + np.sin(x0[0,1]) - 0.3*x0[0,2]**2

# one data set
X = np.random.normal(size=(n, p))
y = 0.5*X[:,0] + np.sin(X[:,1]) - 0.3*X[:,2]**2 \
    + np.random.normal(scale=0.1, size=n)
one = []
for k in ks:
    fit = KNeighborsRegressor(k, weights="uniform", algorithm="brute")
    fit.fit(X, y)
    one.append(fit.predict(x0)[0])

# 200 repetitions
B = 200
pred = np.empty((B, len(ks)))
for b in range(B):
    X = np.random.normal(size=(n, p))
    y = 0.5*X[:,0] + np.sin(X[:,1]) - 0.3*X[:,2]**2 \
        + np.random.normal(scale=0.1, size=n)
    for j, k in enumerate(ks):
        fit = KNeighborsRegressor(k, weights="uniform", algorithm="brute")
        fit.fit(X, y)
        pred[b, j] = fit.predict(x0)[0]
```

```

bias = pred.mean(axis=0) - f0
var = pred.var(axis=0, ddof=0)
mse = ((pred - f0)**2).mean(axis=0)
print(np.max(np.abs(mse - bias**2 - var)))

```

Question 2

a) Seed: 43252.

Setting	KNN MSE	Lasso MSE	Selected λ	Nonzero slopes
Independent	0.5640	0.2344	0.0316	9
Correlated	0.1729	0.2421	0.0316	2

b) With independent predictors, the irrelevant coordinates make Euclidean neighborhoods less informative, while lasso can shrink many slopes to zero. Under strong correlation, the coordinates share information, so distance can better reflect the response-related directions even though KNN still uses all 30 variables.

c) The lasso model is linear, so it cannot directly represent $\sin(X_2)$ or X_3^2 , while KNN can adapt to nonlinear patterns if its neighborhoods are useful. These two data sets do not support a general claim that correlation always helps KNN or that either method is always better.

```

import numpy as np
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Lasso

np.random.seed(43252)
n, p = 400, 30
grid = np.logspace(0, -4, 41)
out = []

for correlated in [False, True]:
    S = np.eye(p) if not correlated else 0.8 + 0.2*np.eye(p)
    Xall = np.random.normal(size=(1400, p)) @ np.linalg.cholesky(S).T
    yall = 0.5*Xall[:,0] + np.sin(Xall[:,1]) - 0.3*Xall[:,2]**2 \
        + np.random.normal(scale=0.1, size=1400)
    X, Xt = Xall[:n], Xall[n:]
    y, yt = yall[:n], yall[n:]

    knn = KNeighborsRegressor(5, algorithm="brute").fit(X, y)
    knn_mse = np.mean((yt - knn.predict(Xt))**2)

    folds = np.random.permutation(np.repeat(np.arange(10), 40))
    cv = np.zeros((10, len(grid)))
    for f in range(10):
        tr, va = folds != f, folds == f
        mu, sd = X[tr].mean(0), X[tr].std(0, ddof=0)
        Xtr, Xva = (X[tr]-mu)/sd, (X[va]-mu)/sd
        ybar = y[tr].mean()
        for j, lam in enumerate(grid):
            fit = Lasso(alpha=lam, fit_intercept=False,
                        tol=1e-8, max_iter=100000)
            fit.fit(Xtr, y[tr]-ybar)
            cv[f,j] = np.mean((y[va]-(fit.predict(Xva)+ybar))**2)

```

```

lam = grid[np.argmin(cv.mean(0))]
mu, sd = X.mean(0), X.std(0, ddof=0)
fit = Lasso(alpha=lam, fit_intercept=False,
            tol=1e-8, max_iter=100000)
fit.fit((X-mu)/sd, y-y.mean())
pred = fit.predict((Xt-mu)/sd) + y.mean()
out.append((knn_mse, np.mean((yt-pred)**2),
            lam, np.count_nonzero(fit.coef_)))

```

Question 3

a) Seed: 43254.

k	Test MSE ($m = 3$)	Test MSE ($m = 30$)
2.0000	1.4974	2.5336
10.0000	1.1396	2.0976
18.0000	1.2110	1.9657
26.0000	1.2857	1.9265
34.0000	1.3450	1.9581
42.0000	1.4052	2.0111
50.0000	1.4771	2.0581
58.0000	1.5514	2.0989
66.0000	1.6092	2.1174
74.0000	1.6726	2.1489
82.0000	1.7326	2.1798

b)

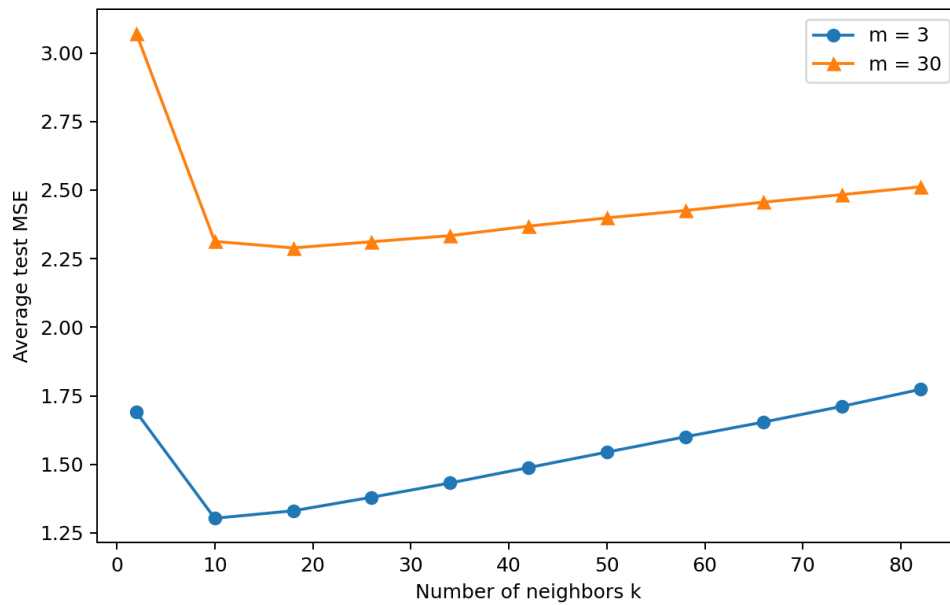


Figure 2: Average test MSE over 50 repetitions.

k	Average MSE ($m = 3$)	Average MSE ($m = 30$)
2.0000	1.6903	3.0711
10.0000	1.3035	2.3130
18.0000	1.3308	2.2892
26.0000	1.3801	2.3118
34.0000	1.4321	2.3339
42.0000	1.4883	2.3686
50.0000	1.5446	2.3987
58.0000	1.6005	2.4256
66.0000	1.6543	2.4561
74.0000	1.7117	2.4835
82.0000	1.7735	2.5120

c) When $m = 3$, most structured variation in the 100 observed variables comes from only a few latent directions; with $m = 30$, many extra latent directions affect distance without affecting Y . The noise η_i also adds variation in all 100 coordinates, so the observed data are not exactly confined to an m -dimensional subspace.

```
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

np.random.seed(43254)
p = 100
ks = np.arange(2, 83, 8)

def one_run(m):
    A = np.random.uniform(-2, 2, size=(p, m))
    Z = np.random.normal(size=(400, m))
    X = Z @ A.T + np.random.normal(size=(400, p))
    y = Z[:,0] + Z[:,1] + np.random.normal(size=400)
    ans = []
    for k in ks:
        fit = KNeighborsRegressor(k, algorithm="brute").fit(X[:200], y[:200])
        ans.append(np.mean((y[200:] - fit.predict(X[200:]))**2))
    return ans

single = np.column_stack([one_run(3), one_run(30)])
runs = np.empty((50, len(ks), 2))
for b in range(50):
    runs[b,:,0] = one_run(3)
    runs[b,:,1] = one_run(30)
mean_mse = runs.mean(axis=0)
```

Question 4

a)

```
import numpy as np

train = np.loadtxt("data/knn/zip-train.csv.gz", delimiter=",")
test = np.loadtxt("data/knn/zip-test.csv.gz", delimiter=",")
X, y = train[:1000,1:], train[:1000,0].astype(int)
Xt, yt = test[:500,1:], test[:500,0].astype(int)

def d_euclid(x, X):
    return np.sqrt(((X-x)**2).sum(axis=1))
```

```

def d_manhattan(x, X):
    return np.abs(X-x).sum(axis=1)

def classify(distance):
    pred = np.empty(len(Xt), dtype=int)
    for i, x in enumerate(Xt):
        d = distance(x, X)
        nbr = np.argsort(d, kind="stable")[:5] # row number breaks ties
        pred[i] = np.argmax(np.bincount(y[nbr], minlength=10))
    return pred

pred_e = classify(d_euclid)
pred_m = classify(d_manhattan)

def confusion(pred):
    C = np.zeros((10,10), dtype=int)
    for p, t in zip(pred, yt):
        C[p,t] += 1
    return C

```

b) The test error was 0.122 for Euclidean distance and 0.138 for Manhattan distance.

Euclidean confusion matrix		0	1	2	3	4	5	6	7	8	9
<i>Rows = predicted, columns = true</i>	0	118	0	7	1	0	4	2	0	2	0
	1	0	66	0	0	2	0	0	0	0	1
	2	0	0	39	1	0	0	2	0	0	1
	3	0	0	0	23	0	4	0	0	2	0
	4	0	0	1	0	27	1	1	1	0	1
	5	0	0	0	1	0	13	0	0	0	0
	6	0	2	0	0	0	1	41	0	2	0
	7	0	0	0	0	1	1	0	34	0	4
	8	0	0	3	2	0	2	0	0	37	0
	9	1	0	0	1	4	0	0	1	1	41

Manhattan confusion matrix		0	1	2	3	4	5	6	7	8	9
<i>Rows = predicted, columns = true</i>	0	118	0	6	1	0	5	3	0	4	0
	1	0	67	2	0	2	0	1	0	0	1
	2	0	0	35	0	0	0	2	0	0	0
	3	0	0	1	24	0	4	0	0	2	0
	4	0	0	1	1	27	1	0	1	0	1
	5	0	0	0	1	0	11	0	0	0	0
	6	0	1	0	0	0	2	40	0	2	0
	7	0	0	2	1	1	0	0	34	0	5
	8	0	0	2	0	0	3	0	0	34	0
	9	1	0	1	1	4	0	0	1	2	41

The most common error was a true 2 classified as 0: 7 times with Euclidean distance and 6 times with Manhattan distance. Other frequent confusions were 5 as 0 or 3, 9 as 7, and 4 as 9.

c)

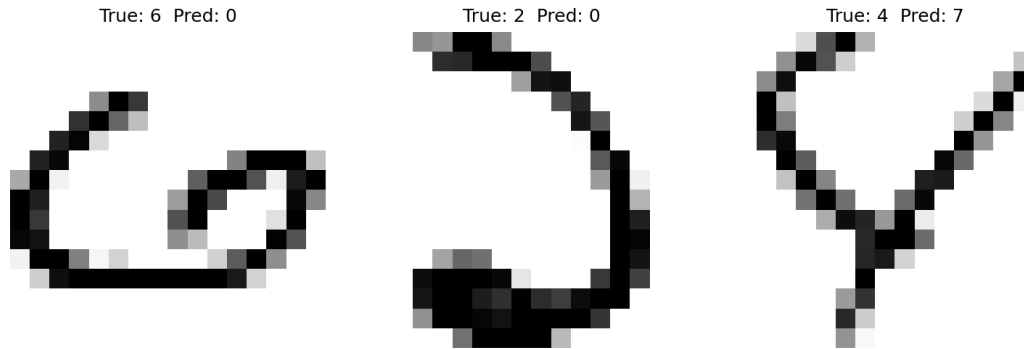


Figure 3: First three Euclidean 5NN errors.

The mistakes are visually understandable because the three images have atypical shapes: the 6 has a rounded loop, the 2 is unusually curved, and the 4 has a weak crossbar. The two methods disagreed on 22 images; Euclidean alone was correct on 11, Manhattan alone on 3, and both were wrong with different predictions on 8, so similar total error can still come from different mistakes.

Question 5

- Created `format-homework-plots/SKILL.md`. The complete final file is appended below.
- The explicit test request was: *“Use format-homework-plots to improve the formatting of my Question 1 plot.”*

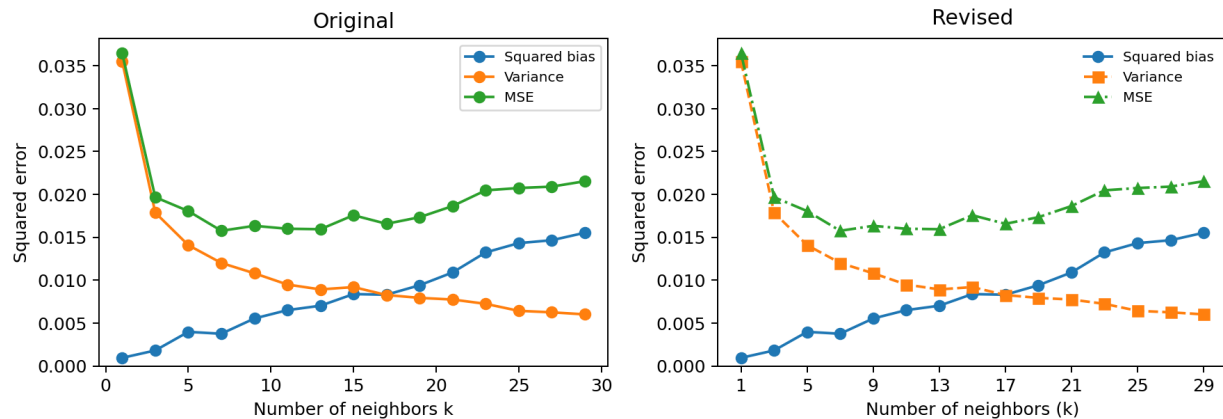


Figure 4: Original Question 1 plot and the version produced after applying the skill rules.

I prefer the revised plot because the title, labels, spacing, markers, and line styles make the three curves easier to distinguish. The statistical content is unchanged, so I kept the skill rules without another revision.

c) Final SKILL.md

```
---
name: format-homework-plots
description: Use only when explicitly requested to improve the formatting of a STAT 432 homework plot.
---
```

STAT 432 Plot Formatting

- Keep the plotted data, calculations, and statistical meaning unchanged.
- Leave enough margin so titles, axes, tick labels, and legends are not clipped.
- Use a short, informative title and clear axis labels with units when needed.
- Keep tick labels readable and avoid crowding.
- Use a colorblind-friendly palette and add different line styles or markers when helpful.
- Start near a 6.5 by 4.5 inch figure with 12-point axis labels and a 14-point title, then adjust if needed.
- Apply the rules to the existing R or Python plot rather than changing the analysis.